

Generative LMs Have Only Partial, Unreliable Script Knowledge

What do Large Language Models Learn about Scripts?
Sancheti & Rudinger (2022)

Presented by David Majekodunmi · CMSC 612

Cloze tests overstate what LMs know about scripts

Scripts: structured knowledge of typical event sequences for everyday scenarios (Schank & Abelson, 1975).

Prior evaluations: fill-in-the-blank cloze tasks the exact format LMs are already optimized for.

The gap: no test of whether LMs can generate a full event sequence unaided, from scratch.

Can pretrained LMs generate complete, ordered event sequences from a scenario name alone?

Given only a scenario prompt (e.g. “bake a cake”), can an LM produce a full, correctly-ordered event sequence description (ESD) without being shown any events in advance?

Contribution: introduces the ESD generation task, plus SIF, a pipeline framework for inducing script knowledge in any LM.

Zero-shot generation is poor: irrelevant, repeated, misordered events

Prompt: *“Here is an ordered sequence of events that occur when you bake a cake:”*

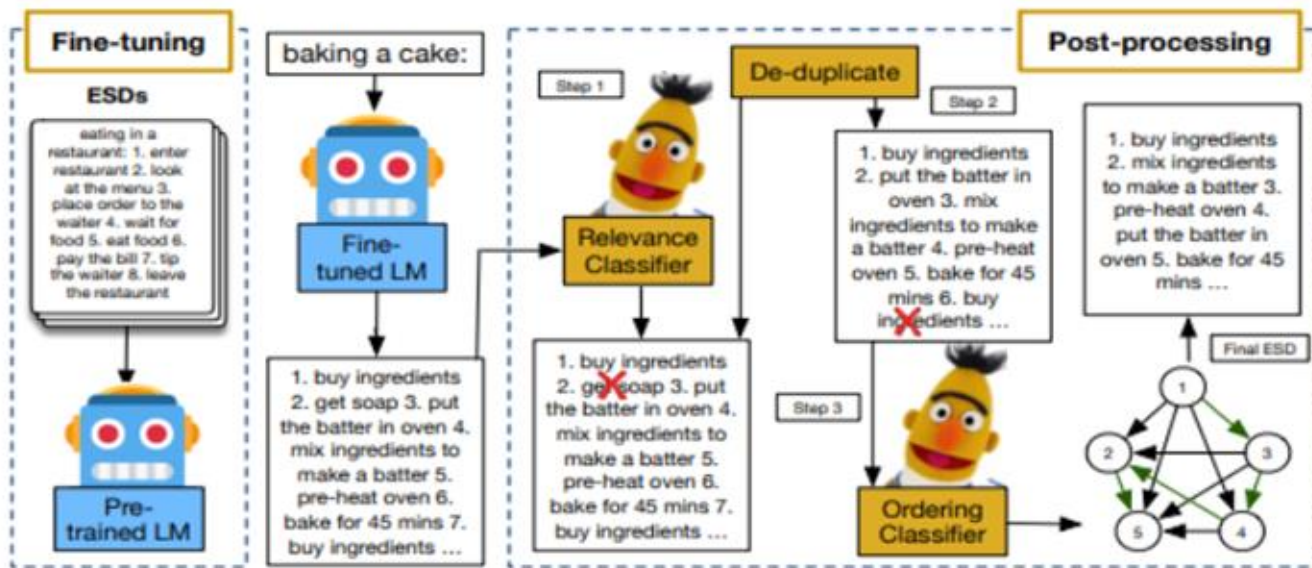
GPT2 output: “3. The cake is served at the table. 4. The dessert is placed on the table.... 6. The cake is transferred to the oven.”

???cake is “served” before it is ever put in the oven???

BART / T5: produced nothing usable in the zero-shot setting.

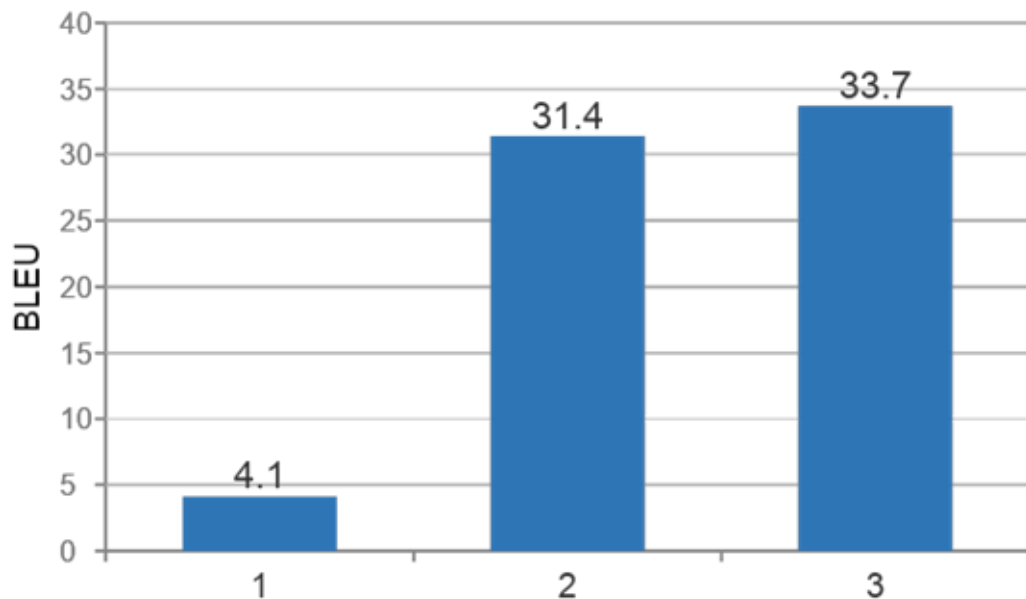
GPT2: produced real attempts, but with irrelevant, repeated, and misordered events.

SIF fixes this with fine-tuning plus a 3-step correction pipeline



Fine-tune → Relevance filter (Step 1) → De-duplicate (Step 2) → Reorder via topological sort (Step 3)

SIF improves BLEU, but the metric doesn't track manual quality



+1–3 BLEU: SIF's post-processing beats plain fine-tuning across prompt variants.

Surprising result: BLEU shows no significant correlation with the authors' own manual quality scores (relevance, ordering, missingness).

$r \approx 0.23$ (relevance), -0.06 (ordering), -0.49 (missingness) — all $p > 0.1$.

SIF's staged design outperforms single-pass filtering methods

Diagnosable errors: staged classifiers isolate which error is fixed (relevance vs. duplication vs. ordering) unlike "Distilling Script Knowledge from Large Language Models for Constrained Language Planning" (Yuan et al., 2023), whose single-pass over-generate-then-filter approach doesn't let you diagnose failure type.

Flexible output: generates fully free-text ESDs, not constrained to a fixed event vocabulary more flexible than "proScript: Partially Ordered Scripts Generation via Pre-trained Language Models" (Sakaguchi et al., 2021), whose graph-based approach depends on a predefined event/edge schema.

Strict full ordering ignores genuinely flexible event sequences

Rigid ordering: forces a total order even for scenarios with genuinely parallel steps (e.g., wet vs. dry ingredients while baking) — proScript's (Sakaguchi et al., 2021) partial-order graphs solve exactly this.

Narrow training data: fine-tuned on 40 scenarios from one crowdsourced dataset — vs. Wadhwa et al.'s (2024) KG-guided approach, which needs no curated ESD set at all.

Shallow de-duplication: only removes exact repeats, not paraphrased ones (authors' own stated limitation).

The relevance → dedup → order pattern generalizes beyond everyday scripts

Instructional video step extraction: filter irrelevant frames/steps, remove repeated demonstrations, enforce correct order.

Customer-support troubleshooting flows: generate correct diagnostic steps, drop irrelevant branches, preserve dependency order.

Onboarding & procedure documentation: same “right steps, right order, no extras” requirement, in an enterprise domain instead of everyday tasks.

Conclusions

- 1. Partial, unreliable script knowledge:** pretrained LMs know some script structure zero-shot, but unevenly — GPT2 far outperforms BART/T5.
- 2. SIF helps, with real limits:** the staged pipeline improves quality, but forces rigid ordering and leans on a BLEU metric that doesn't track manual judgments.
- 3. The pattern generalizes:** relevance-filter → de-duplicate → reorder extends well past cooking and errands — to video, support, and documentation tasks.

References

Chambers, N. & Jurafsky, D. (2008). Unsupervised learning of narrative event chains. Proceedings of ACL-08: HLT, 789–797.

Sakaguchi, K., Bhagavatula, C., Le Bras, R., Tandon, N., Clark, P., & Choi, Y. (2021). proScript: Partially ordered scripts generation via pre-trained language models. arXiv:2104.08251.

Sancheti, A. & Rudinger, R. (2022). What do Large Language Models Learn about Scripts? Proceedings of the 11th Joint Conference on Lexical and Computational Semantics (*SEM 2022), 1–11.

Schank, R. C. & Abelson, R. P. (1975). Scripts, plans, and knowledge. Proceedings of IJCAI, 75, 151–157.

Wadhwa, S., Hassanzadeh, O., Bhattacharjya, D., Barker, K., & Ni, J. (2024). Distilling event sequence knowledge from large language models. Proceedings of ISWC 2024, LNCS 15231.

Yuan, S., Chen, J., Fu, Z., Ge, X., Shah, S., Jankowski, C., Xiao, Y., & Yang, D. (2023). Distilling script knowledge from large language models for constrained language planning. Proceedings of ACL 2023.